

Phase 5 Documentation - Frontend Deployment (Vercel + Custom Domain)

Objective

Deploy the React (Vite) frontend on Vercel, connect it with the Render backend, configure environment variables, fix deployment issues, and attach a custom domain from Hostinger.

Prerequisites

- Backend successfully deployed on Render.
- MongoDB Atlas connected.
- GitHub repository updated.
- Render Health API working.
- Frontend working locally.

Step 1 - Verify Frontend API Configuration

Checked `src/api/client.js`.

The project already contained:

```
export const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000";
```

Conclusion:

- No code change required.
- Local development uses localhost.
- Production uses `VITE_API_URL` supplied by Vercel.

Step 2 - Create Local Environment File

Created:

`frontend/.env`

Content:

```
VITE_API_URL=http://localhost:5000
```

Purpose:

To make the local frontend communicate with the local backend.

Step 3 - Test Local Project

Started backend:

```
npm run dev
```

Started frontend:

```
npm install  
npm run dev
```

Verified:

- Frontend loaded.
- Backend connected.
- MongoDB connected.
- APIs working correctly.

Step 4 - Push Latest Code to GitHub

Commands:

```
git status  
git add .  
git commit -m "Prepare frontend deployment"  
git push origin master
```

Step 5 - Git Problem Encountered

Problem:

Vercel could not detect the frontend folder.

Investigation:

```
git ls-tree HEAD
```

Output:

```
160000 frontend
```

Meaning:

Git was treating frontend as a Submodule instead of a normal folder.

Solution:

```
git rm --cached frontend  
git add frontend
```

```
git commit -m "Fix frontend submodule issue"
git push origin master
```

Result:

Frontend became a normal Git folder and Vercel detected it correctly.

Step 6 - Deploy Frontend on Vercel

Configuration:

Framework: Vite

Root Directory: frontend

Build Command: npm run build

Output Directory: dist

Install Command: npm install

Environment Variable:

Name:

VITE_API_URL

Value:

<https://bitu-website.onrender.com>

Deployment completed successfully.

Step 7 - Connect Custom Domain

Domain Provider:

Hostinger

Domain:

minimalhost.online

Steps:

1. Added domain inside Vercel.
2. Vercel displayed required DNS records.
3. Opened Hostinger DNS Zone.
4. Updated A Record.
5. Updated www CNAME.
6. Waited for DNS propagation.
7. Vercel changed status from Invalid Configuration to Valid Configuration.
8. Website became accessible through the custom domain.

Problems Faced and Solutions

Problem	Reason	Solution
Frontend using localhost	Production cannot use localhost	Used VITE_API_URL.
Confusion about client.js	Thought API URL should be hardcoded	No changes required.
Frontend not detected in Vercel	Git Submodule	Removed submodule and re-added folder.
Root Directory issue	Repository contained backend and frontend	Selected frontend as Root Directory.
Invalid Domain Configuration	DNS records mismatched	Updated Hostinger DNS according to Vercel.
DNS delay	Propagation takes time	Waited until Valid Configuration appeared.

What I Learned

- Difference between localhost and production deployment.
- How VITE_API_URL works.
- Why Environment Variables are used.
- How Vercel deploys a Vite application.
- Why Root Directory matters in monorepos.
- Difference between Git Submodule and a normal folder.
- How to debug deployment issues using Git and Vercel logs.
- How DNS, A Records and CNAME records work.
- How to connect a Hostinger domain with Vercel.
- Complete frontend deployment workflow.

Final Result

Frontend: Vercel

Backend: Render

Database: MongoDB Atlas

Domain: minimalhost.online

HTTPS: Enabled

Status: Successfully Deployed